READ FIRST · UNPACK THE ABBREVIATIONS

CPU (Central Processing Unit)

RAM (Random Access Memory)

SSD (Solid-State Drive)

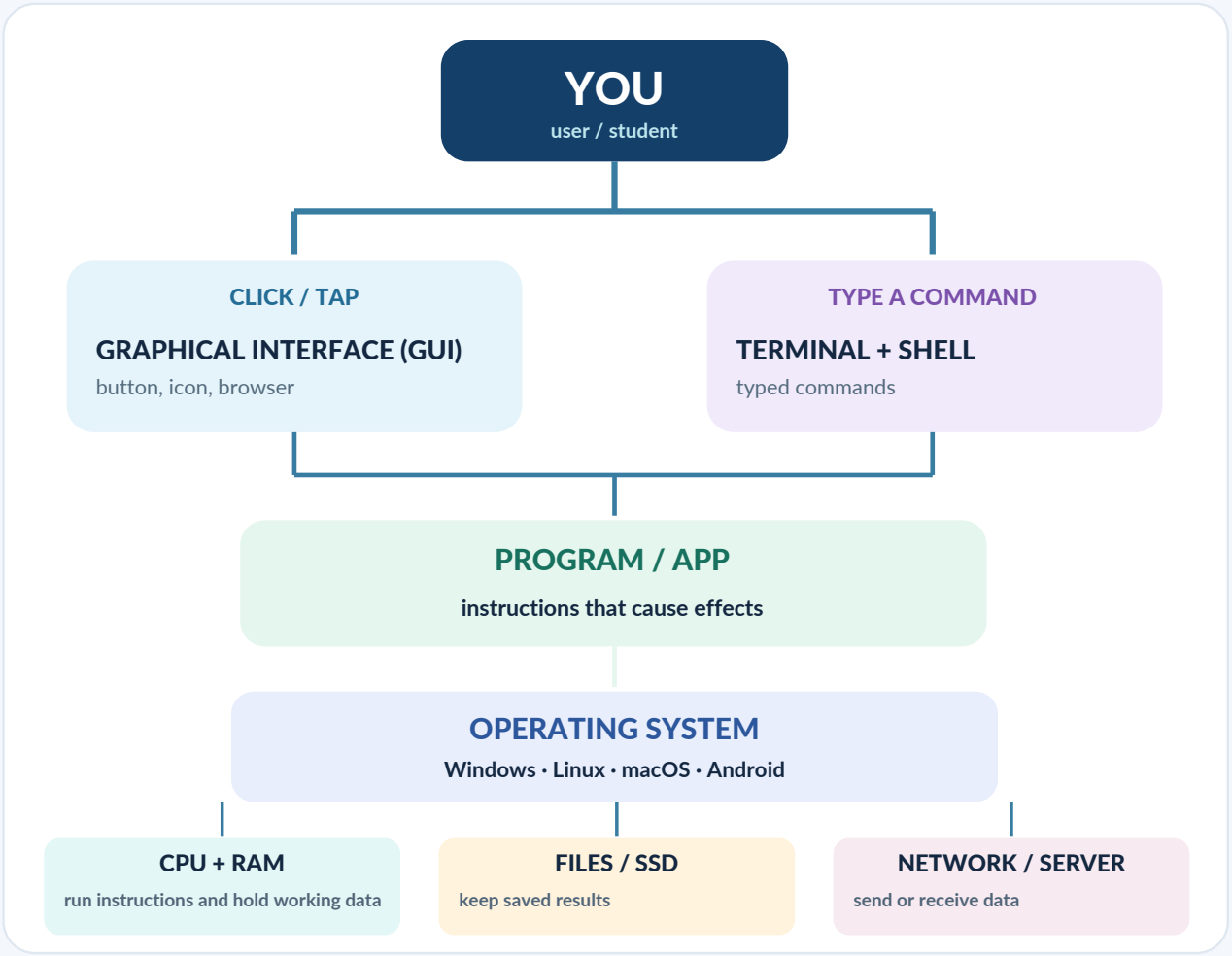
GUI (Graphical User Interface)

COMPUTER SCIENCE / YEAR 1 / VISUAL MASTER MAP

YOUR COMPUTER: THE MAP

One machine. Two ways to tell it what you want. A visible result.

SCAN & LEARN



WHY LEARN DATA STRUCTURES?

STRUCTURES = ORGANIZE

Where do we put information?

ALGORITHMS = STEPS

What do we do with information?

BIG-O = GROWTH

How much extra work appears?

REMEMBER

From a click or command to a result: DATA + INSTRUCTIONS + EXECUTION.

Remember: the terminal and the browser are not separate computers; they are apps that can use the same resources.

READ FIRST · UNPACK THE ABBREVIATIONS

CPU (Central Processing Unit)

OS (Operating System)

COMPUTER SCIENCE / DATA STRUCTURES / YEAR 1

THE COMPUTER, EXPLAINED

A phone, laptop and desktop computer share the same basic idea.

SCAN & LEARN

WHAT HAPPENS • FOLLOW THE ARROWS

01 YOU PRESS A BUTTON → 02 THE APP REQUESTS → 03 THE OS COORDINATES → 04 HARDWARE WORKS

FOUR IDEAS TO KEEP

1 HARDWARE = physical parts

Things you can touch: screen, processor, memory, storage. The processor (CPU) executes instructions.

Picture a kitchen with its tools and appliances.

2 SOFTWARE = instructions

Apps are programs: browser, game, camera. Code tells them which operations to perform.

Picture the recipe used by a cook.

3 OS = the coordinator

Windows, Linux, macOS and Android are operating systems. They manage apps, files and devices.

The OS is NOT the browser. The browser is an app.

4 INPUT → PROCESS → OUTPUT

Input: click or key. Processing: instructions run. Output: an image, sound or saved file.

Example: press Volume Up → the sound changes.

TRY IT — NO COMPUTER NEEDED

One student is the user, one an app, one the OS, one the CPU. Act out pressing “Play”.

THE ONE SENTENCE TO REMEMBER

The SCREEN shows the result; apps, the OS and hardware do the work.

Question: Is Android hardware, an app or an operating system?

READ FIRST · UNPACK THE ABBREVIATIONS

URL (Uniform Resource Locator)

DNS (Domain Name System)

HTML (HyperText Markup Language)

CSS (Cascading Style Sheets)

Wi-Fi (wireless-network technology; not an acronym)

COMPUTER SCIENCE / DATA STRUCTURES / YEAR 1

BROWSERS AND THE INTERNET

Why a website is not simply “inside” your browser.

SCAN & LEARN

WHAT HAPPENS • FOLLOW THE ARROWS

01 ENTER AN ADDRESS → 02 FIND THE SERVER → 03 REQUEST THE FILES → 04 DISPLAY THE PAGE

FOUR IDEAS TO KEEP

1 BROWSER ≠ INTERNET

Chrome, Firefox and Safari are apps. The internet is a network connecting devices. The web is one service that uses it.

The browser is a window; the internet is the road.

2 ADDRESS ≠ SEARCH

A web address (URL) says where to request a resource. A search engine, such as Google, helps find possible addresses.

site.org/class is an address; “learn Python” is a search.

3 WHO SENDS THE PAGE?

Your browser (client) requests data from a server. DNS helps find its network address. The server responds; the browser renders.

404 = the requested resource was not found.

4 WHAT MAKES A WEBSITE?

HTML = structure. CSS = appearance. JavaScript = interactive behavior. The browser assembles them into a visible page.

A visible button can trigger code when clicked.

TRY IT — A PLAY WITHOUT WI-FI

One student is the browser, one DNS, one the server. Request a web page drawn on paper.

THE ONE SENTENCE TO REMEMBER

A BROWSER is an app that REQUESTS, RECEIVES and DISPLAYS pages.

Question: Can you open a local HTML file without internet?

READ FIRST · UNPACK THE ABBREVIATIONS

GUI (Graphical User Interface)

ADB (Android Debug Bridge)

OS (Operating System)

PC (Personal Computer)

COMPUTER SCIENCE / DATA STRUCTURES / YEAR 1

TERMINAL: A DIFFERENT DOOR

Same machine: clicking icons or typing instructions.

SCAN & LEARN

WHAT HAPPENS • FOLLOW THE ARROWS

01 TYPE A COMMAND → 02 THE SHELL READS IT → 03 THE OS DOES WORK → 04 SEE THE RESULT

FOUR IDEAS TO KEEP

1 WHAT IS A TERMINAL?

An app where you type commands. The “shell” is the program interpreting them. They are not the same thing.

Window = terminal; interpreter = shell.

2 WINDOWS

Start → search “Terminal” or “PowerShell”. Windows Terminal can host PowerShell or Command Prompt; their commands may differ.

PowerShell: pwd (where am I?), ls (what is here?).

3 LINUX / macOS

Ubuntu: Ctrl+Alt+T or search Terminal. macOS: Spotlight → Terminal or Applications → Utilities.

In the terminal: pwd, ls, cd folder_name.

4 WHAT ABOUT ANDROID?

Android also has an OS and shell, but ordinary users do not get a standard built-in terminal app. Developers use special tools.

From a PC, ADB can open an Android shell; not needed yet.

TRY IT

PREDICT THE EFFECT

Teacher types “pwd” and “ls”. Students predict what appears before Enter is pressed.

THE ONE SENTENCE TO REMEMBER

GUI = SHOW what you want by clicking. TERMINAL = TYPE what you want.

Question: How are a terminal and a shell different?

READ FIRST · UNPACK THE ABBREVIATIONS

GUI (Graphical User Interface)

HTML (HyperText Markup Language)

HTTP (Hypertext Transfer Protocol)

COMPUTER SCIENCE / DATA STRUCTURES / YEAR 1

FROM TERMINAL TO SCREEN

A demonstration with ONE COMPUTER for an entire class.

SCAN & LEARN

WHAT HAPPENS • FOLLOW THE ARROWS

01 CREATE A FOLDER → 02 SEE IT IN THE GUI → 03 ADD A WEB PAGE → 04 OPEN IN BROWSER

FOUR IDEAS TO KEEP

1 1. CREATE SOMETHING VISIBLE

Open Terminal/PowerShell. Type “mkdir demo”. It makes a folder here. Type “cd demo”, then inspect the new folder using File Explorer / Files.

“pwd” shows the location; “ls” lists its contents.

2 2. WRITE A WEB PAGE

In the demo folder, create index.html using a plain-text editor. Type the exact code shown below; in Notepad choose “All files” when saving, not .txt.

<h1>Hello, world!</h1> → a large heading on screen.

3 3. MAKE IT APPEAR

Open index.html in your browser. Change “Hello” to “Welcome”, save, then reload. The code changed what you see!

A local HTML file works even without the internet.

4 4. OPTIONAL: LOCAL SERVER

If Python is installed, run a local server in the demo folder and open http://127.0.0.1:8000/. Stop it with Ctrl+C.

Windows: py -m http.server 8000 --bind 127.0.0.1
Linux/Mac: python3 -m http.server 8000 --bind 127.0.0.1

TRY IT — ONE-DEVICE CLASS DEMO

Before every Enter, ask: “What will change? Where can we see that change?”.

THE ONE SENTENCE TO REMEMBER

The TERMINAL can change files; the BROWSER can display those files as a page.

Caution: Python’s built-in server is for local demos, not production.

HOW TO READ CODE

Reading code means tracking steps, not memorizing a language.

SCAN & LEARN

WHAT HAPPENS • FOLLOW THE ARROWS

01 READ THE LINE → 02 EVALUATE EXPRESSION → 03 FOLLOW THE VALUE → 04 OBSERVE THE OUTPUT

FOUR IDEAS TO KEEP

1 VARIABLE = A NAME TAG

"score = 3" assigns 3 to the name score. You can assign a new value later. "=" assigns; "==" tests equality.

score = 3 → score = score + 1 → score is now 4.

2 FUNCTION = A MINI-RECIPE

"len" finds length; "print" displays output. In nested calls, you need the inner call result before the outer call can finish.

print(len("Ana")) → len("Ana") = 3 → print(3).

3 NOT ALWAYS RIGHT-TO-LEFT!

There is NO general rule to read code right-to-left. Follow execution order; Python generally evaluates expressions left-to-right.

Only the INNER call must finish before its outer call.

4 IF, FOR AND INDENTATION

"if" chooses a path; "for" repeats. Leading spaces form the code blocks in Python.

if score > 3: → run the indented block if true.

TRY IT — TRACE WITHOUT RUNNING

Write: x = 2; x = x + 3; print(x). Ask everyone to predict what gets printed.

THE ONE SENTENCE TO REMEMBER

CODE changes the current STATE: "what value exists RIGHT NOW?"

Question: What does print(len("cat")) display?

ALGORITHMS AND DEBUGGING

How to solve a problem systematically without a computer.

SCAN & LEARN

WHAT HAPPENS • FOLLOW THE ARROWS

01 GET A TASK → 02 CHOOSE CLEAR STEPS → 03 FOLLOW THE STEPS → 04 CHECK THE RESULT

FOUR IDEAS TO KEEP

1 ALGORITHM = PRECISE RECIPE

A finite set of clear steps for a problem. Input is what comes in; output is what comes out. Instructions must be unambiguous.

Example: given 3 numbers → find the largest.

2 DECISIONS AND LOOPS

Decision: “if raining, take an umbrella”. Loop: “for each book, read its title”. Both are building blocks of programs.

“If...” = condition. “For each...” = loop.

3 USE A TRACE TABLE

When code changes values, write the current state after each step. Do not guess by glancing at the code.

step 0: x=1 | step 1: x=2 | step 2: x=4.

4 THREE KINDS OF BUG

Syntax: invalid instruction. Runtime: the program crashes. Logic: it runs but gives a wrong answer.

Debug: reproduce → find the first wrong step → check.

TRY IT — HUMAN ROBOT

Tell a classmate how to draw a square without pointing. Which instructions were missing?

THE ONE SENTENCE TO REMEMBER

A program is not “magic”: you can SIMULATE it one step at a time.

Question: Wrong answer, no crash = what kind of error?

READ FIRST · UNPACK THE ABBREVIATIONS

LIFO (Last In, First Out)

FIFO (First In, First Out)

COMPUTER SCIENCE / DATA STRUCTURES / YEAR 1

DATA STRUCTURES — I

Information needs organization, like objects in real life.

SCAN & LEARN

WHAT HAPPENS • FOLLOW THE ARROWS

01 YOU HAVE VALUES → 02 ORGANIZE THEM → 03 CHOOSE A RULE → 04 FIND THEM EASILY

FOUR IDEAS TO KEEP

1 LIST = ITEMS IN ORDER

Like seats on a train: each item has a position. In Python the first position is 0, not 1. You can visit or change items.

["Ana", "Dan", "Mia"] → position 0 is “Ana”.

2 STACK = STACK OF PLATES

Add on top, remove from the top. Last In, First Out: LIFO. Useful for Undo and function calls.

Push A, then B, then C → pop C first.

3 QUEUE = CHECKOUT LINE

First In, First Out: FIFO. Items join at the back and leave from the front. Useful for waiting tasks.

A, B, C join → A leaves first.

4 SET = UNIQUE ITEMS

Stores values without duplicates. Their order is not guaranteed. Useful for “Have I seen this name before?”.

[A, B, A] in a set → {A, B}.

TRY IT — A CLASS WITHOUT SCREENS

Three students are A, B, C. Use where you stand to show removal order for a stack and queue.

THE ONE SENTENCE TO REMEMBER

LIST = position · STACK = last · QUEUE = first · SET = no duplicates.

Question: Which structure resembles the Undo button?

DATA STRUCTURES — II

How we look things up and represent connections.

SCAN & LEARN

WHAT HAPPENS • FOLLOW THE ARROWS

01 WHAT IS THE TASK? → 02 PICK A STRUCTURE → 03 PERFORM AN ACTION → 04 CONSIDER THE COST

FOUR IDEAS TO KEEP

1 DICTIONARY = KEY → VALUE

Look up an item using a KEY, not a position. Examples: name → phone; product → price. Called dict in Python.

{"Ana": 12, "Dan": 9} → key “Ana” gives 12.

2 TREE = FAMILY OF BRANCHES

Each node (except the root) has one parent. Computer folders are often organized like a tree.

University → Year 1 → Semester 1 → Lab.

3 GRAPH = DOTS AND LINKS

Nodes connected by edges. Friend networks and metro routes are graphs. Graphs can have cycles.

A — B — C and A — C: more than one path exists.

4 HOW DO YOU CHOOSE?

Order → list. Oldest first → queue. Newest first → stack. Lookup by key → dict. Relations → graph.

The right structure simplifies a program and can save work.

TRY IT — DRAW CONNECTIONS

On paper: draw 5 towns and roads between them. Can you reach any town from any other?

THE ONE SENTENCE TO REMEMBER

A DATA STRUCTURE is the RULE for storing and finding information.

Question: What structure fits “student name → grade”?

READ FIRST · UNPACK THE ABBREVIATIONS

RAM (Random Access Memory)

SSD (Solid-State Drive)

COMPUTER SCIENCE / DATA STRUCTURES / YEAR 1

FILES, MEMORY, DATA

Where information “lives” and why it can disappear.

SCAN & LEARN

WHAT HAPPENS • FOLLOW THE ARROWS

01 THE APP IS RUNNING → 02 WORK IS HELD IN RAM → 03 YOU PRESS “SAVE” → 04 THE FILE PERSISTS

FOUR IDEAS TO KEEP

1 RAM ≠ SSD / DRIVE

RAM holds active working data temporarily. SSD/drive preserves files after shutdown. “Save” writes data for later use.

An unsaved document may disappear on shutdown.

2 FILE, FOLDER, PATH

A file holds data stored under a name. A folder groups files. A path tells where a file is inside folders.

Example: Documents / course / assignment.txt.

3 BIT, BYTE, TEXT

A computer stores information as bits (0 or 1). 8 bits make one byte. Text, images and sound are encoded as data.

The letter “A” gets a number using a text encoding.

4 EXTENSIONS AND BACKUPS

.txt, .jpg and .py suggest file types, but do not prove their contents. Backups protect your work.

“In the cloud” usually means on another computer (server).

TRY IT — DRAW A FILE PATH

On paper: university → Year 1 → lab → task.py. Which is the file? Which are folders?

THE ONE SENTENCE TO REMEMBER

RAM = worktable · SSD = storage cabinet · FILE = saved item.

Question: What do you risk if you shut down without saving?

READ FIRST · UNPACK THE ABBREVIATIONS

AI (Artificial Intelligence)

MFA (Multi-Factor Authentication)

HTTPS (Hypertext Transfer Protocol Secure)

QR (Quick Response)

COMPUTER SCIENCE / DATA STRUCTURES / YEAR 1

CYBER SAFETY + SMART AI USE

Good habits from your very first day.

SCAN & LEARN

WHAT HAPPENS • FOLLOW THE ARROWS

01 PAUSE → 02 CHECK THE SOURCE → 03 LIMIT ACCESS → 04 VERIFY THE RESULT

FOUR IDEAS TO KEEP

1 PROTECT YOUR ACCOUNTS

Use long, unique passwords with a manager; prefer passkeys when offered. Enable multifactor authentication (MFA).

Never share MFA or account recovery codes.

2 LINKS, FILES, QR CODES

Urgent messages may be scams. Check the real domain, sender and destination before clicking. QR codes can also hide URLs.

The HTTPS padlock does not prove a site is honest.

3 TERMINAL COMMANDS HAVE POWER

Do not paste unknown commands or run untrusted scripts. Administrator/root rights are powerful; grant only what is needed.

Do not disable protection just because a tutorial says so.

4 AI SUGGESTS; YOU VERIFY

AI can explain but also invent facts or generate unsafe code. Ask for tests; verify outputs; never paste secrets or private records.

For any solution: understand → test → verify.

TRY IT — PHISHING ROLE-PLAY

“Your account expires today. Log in now at login-uni.example.” What do you check first?

THE ONE SENTENCE TO REMEMBER

ASK YOURSELF: “Do I understand it? Have I verified it? Can I undo it?”

Question: Is it safe to execute AI-generated code without checking it?

NO MATH OR CODING REQUIRED

BIG-O

SAY IT
“BIG O”

SCAN & LEARN

HOW MUCH MORE WORK when there are MORE things?

4 ITEMS → 8 ITEMS — Twice as many!

ONE STEP
one small action you carry out

n
how many items to check

O(...)
how work grows

SAME 4 → 8 ITEMS. SIX WAYS THE AMOUNT OF WORK CAN GROW:

O(1) STAYS THE SAME

I know which drawer. I open it, however many drawers exist.

straight to known spot

4 items — 1 step → 8 items — 1 step

O(log n) GROWS VERY LITTLE

Search an ORDERED list. Discard half in each round.

4 › 2 › 1 — 2 rounds
8 › 4 › 2 › 1 — 3 rounds

4 items — 2 rounds → 8 items — 3 rounds

O(n) DOUBLES

Search drawers ONE BY ONE. If it is last, check all drawers.

1 2 3 4 →
1 2 3 4 5 6 7 8 →

4 items — 4 checks → 8 items — 8 checks

O(n log n) MORE THAN DOUBLES

Sort cards: split them into groups, then merge in order.

→ 2 rounds (of work)
→ 3 rounds (of work)

4 items — ~8 steps* → 8 items — ~24 steps*

O(n²) QUADRUPLES

Compare each item with EVERY item (including itself).

4 × 4 = 16 → 8 × 8 = 64

4 items — 16 checks → 8 items — 64 checks

O(2ⁿ) EXPLODES

For each item choose YES or NO. Try ALL possible combinations.

YES / NO — YES / NO

4 items — 16 ways → 8 items — 256 ways

REMEMBER JUST ONE QUESTION:

“If I double the items, HOW MUCH MORE WORK?”

same · a little · ×2 · over ×2 · ×4 · explodes

Big-O is not seconds or exact steps: it bounds how the workload grows.

* O(n log n) values show a simplified model, not measured steps.

QUICK REFERENCE • YEAR 1

BIG -O

HOW DOES THE WORK GROW WHEN THE DATA GROWS?

n = number of items



SCAN & LEARN

MENTAL EXPERIMENT

10 → 20

DOUBLE THE INPUT (n).

What happens to the number of operations?

Compare the model values in the cards.

THE SIX PATTERNS TO RECOGNIZE

EASIER → HARDER TO SCALE

O(1)

CONSTANT

Direct access; independent of n.



`a[0]` 1 → 1

O(log n)

LOGARITHMIC

Cut the problem in half.



`n //= 2` 3.3 → 4.3

O(n)

LINEAR

Visit each item once.



`for x in a:` 10 → 20

O(n log n)

N × LOG N

Split, then merge.

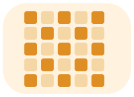


`merge_sort(a)` 33 → 86

O(n²)

QUADRATIC

Compare each with every item.



2 nested loops 100 → 400

O(2ⁿ)

EXPONENTIAL

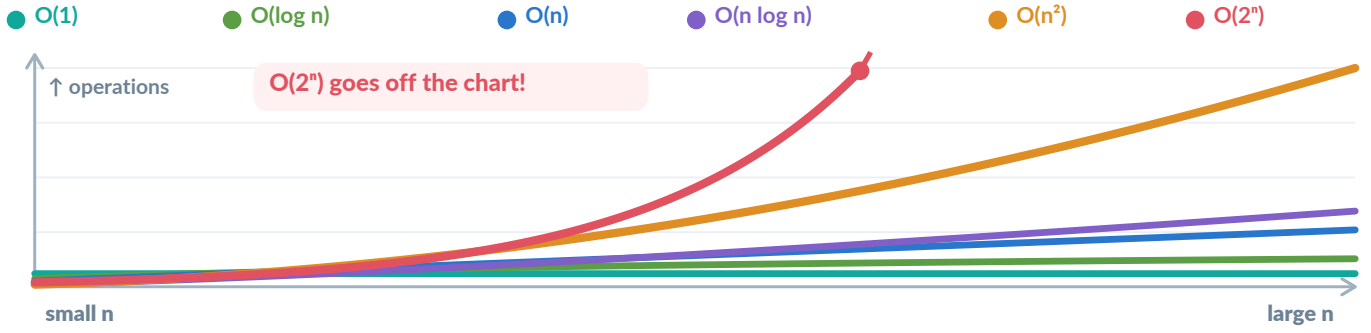
Try every possible subset.



all subsets 1 024 → 1 048 576

GROWTH CURVES: AS n GROWS →

shape matters, not seconds



THREE SURVIVAL RULES

01 LOOPS ONE AFTER ANOTHER

$O(n) + O(n) = O(n)$

add; keep dominant term

02 LOOPS INSIDE LOOPS

$O(n) \times O(n) = O(n^2)$

multiply repetitions

03 DROP CONSTANT FACTORS

$O(3n^2 + 7n + 2) = O(n^2)$

for large n, n^2 dominates

REMEMBER

Big-O = an upper growth bound, not runtime in seconds.

O(1) ≠ exactly one step. Specify the case and resource: time / memory.